HW 4

1. PLU factorization of

$$A = \begin{pmatrix} 0 & 3 & 3 & -1 \\ 1 & 2 & -1 & 0 \\ 2 & 4 & -2 & 1 \\ -1 & 1 & 0 & 2 \end{pmatrix} \qquad PA = LU$$

$$P = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} & & 1 & \\ & 1 & & \\ 1 & & & \\ & & & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} & & 1 & \\ 1 & & & \\ & 1 & & \\ & & & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} & & 1 & \\ 1 & & & \\ & & & 1 \\ & 1 & & \end{bmatrix}$$

swap 1 and 3   swap 2 and 3   swap 3 and 4

$$\begin{pmatrix} 2 & 4 & -2 & 1 \\ 1 & 2 & -1 & 0 \\ 0 & 3 & 3 & -1 \\ -1 & 1 & 0 & 2 \end{pmatrix} \quad \begin{pmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ 1 & 2 & -1 & 0 \\ -1 & 1 & 0 & 2 \end{pmatrix} \quad \begin{pmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ -1 & 1 & 0 & 2 \\ 1 & 2 & -1 & 0 \end{pmatrix}$$

$$PA = \begin{pmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ -1 & 1 & 0 & 2 \\ 1 & 2 & -1 & 0 \end{pmatrix} \underset{=}{\overset{R_3 - (-\frac{1}{2})R_1}{R_4 - \frac{1}{2}R_1}} \begin{pmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ 0 & 3 & -1 & 5/2 \\ 0 & 0 & 0 & -\frac{1}{2} \end{pmatrix} \overset{R_3 - R_2}{=} \begin{pmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ 0 & 0 & -4 & 7/2 \\ 0 & 0 & 0 & -1/2 \end{pmatrix}$$

$$\boxed{\underset{P}{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}} \cdot A = \underset{L}{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -1/2 & 1 & 1 & 0 \\ 1/2 & 0 & 0 & 1 \end{bmatrix}} \underset{U}{\begin{bmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ 0 & 0 & -4 & 7/2 \\ 0 & 0 & 0 & -1/2 \end{bmatrix}}}$$

Solve $Ax = b$, $b = (5\ 0\ 1\ 4)^T$

$$Ax = b \Rightarrow PAx = Pb \Rightarrow LUx = Pb$$

$$Pb = b' = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 0 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 4 \\ 0 \end{bmatrix}$$

$$Ly = b' = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -1/2 & 1 & 1 & 0 \\ 1/2 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 4 \\ 0 \end{bmatrix}$$

$y_1 = 1$

$y_2 = 5 - 0(1) = 5$

$y_3 = 4 - (-\frac{1}{2})(1) - 1(5) = 4 + \frac{1}{2} - 5 = -\frac{1}{2}$

$y_4 = 0 - \frac{1}{2}(1) = -\frac{1}{2}$

$$Ux = y = \begin{bmatrix} 2 & 4 & -2 & 1 \\ 0 & 3 & 3 & -1 \\ 0 & 0 & -4 & 7/2 \\ 0 & 0 & 0 & -1/2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ -1/2 \\ -1/2 \end{bmatrix}$$

$-\frac{1}{2}x_4 = -\frac{1}{2} \Rightarrow x_4 = 1$

$-4x_3 + \frac{7}{2}x_4 = -\frac{1}{2} \Rightarrow -4x_3 = -4 \Rightarrow x_3 = 1$

$3x_2 + 3x_3 - x_4 = 5 \Rightarrow 3x_2 = 5 - 3 + 1 \Rightarrow x_2 = 1$

$2x_1 + 4x_2 - 2x_3 + x_4 = 1 \Rightarrow 2x_1 = 1 - 4 + 2 - 1 \Rightarrow x_1 = -1$

$$\boxed{x = \begin{pmatrix} -1 \\ 1 \\ 1 \\ 1 \end{pmatrix}}$$

2. $L_{ij}(\theta)$ is $n \times n$ unit lower triangular $\begin{bmatrix} 1 & & 0 \\ & 1 \ddots & \\ \theta & & 1 \end{bmatrix}$

a) Verify $L_{ij}(\theta)^{-1} = L_{ij}(-\theta)$

To prove the above, it is equivalent to prove

$$L_{ij}(\theta) \cdot L_{ij}(-\theta) = I$$

Using the definition of matrix multiplication,

$$\left(L_{ij}(\theta) \cdot L_{ij}(-\theta)\right)_{ij} = \sum_{k=1}^{n} \left(L_{ij}(\theta)\right)_{ik} \left(L_{ij}(-\theta)\right)_{kj}$$

where the only nonzero terms are the diagonal and the θ position. Thus,

$$\text{Entry at } i,j = \left(L_{ij}(\theta)\right)_{ii} \cdot \left(L_{ij}(-\theta)\right)_{ij} + \left(L_{ij}(\theta)\right)_{ij} \cdot \left(L_{ij}(-\theta)\right)_{jj}$$

$$= 1 \cdot (-\theta) + (\theta) \cdot 1 = -\theta + \theta = 0$$

Because the entry at i,j is 0, the product is indeed the identity matrix and $L_{ij}(\theta)^{-1} = L_{ij}(-\theta)$. In row operations language, this multiplication is equivalent to scaling row j by θ and adding it to row i. The inverse is scaling row j by $-\theta$ and adding to row i.

b) $L_{ij}(\theta_1) \cdot L_{kj}(\theta_2)$, $k \neq i$

The result will look unit lower triangular with both θ_1 and θ_2 at their respective spots. This is because during row operations, each operation is independent of each row due to row j being a row of the identity.

3. a) Write down every 3x3 permutation matrix

$$\begin{pmatrix}1&0&0\\0&1&0\\0&0&1\end{pmatrix} \begin{pmatrix}1&0&0\\0&0&1\\0&1&0\end{pmatrix} \begin{pmatrix}0&1&0\\1&0&0\\0&0&1\end{pmatrix} \begin{pmatrix}0&1&0\\0&0&1\\1&0&0\end{pmatrix} \begin{pmatrix}0&0&1\\1&0&0\\0&1&0\end{pmatrix} \begin{pmatrix}0&0&1\\0&1&0\\1&0&0\end{pmatrix}$$

b) Show that the inverse of a permutation matrix is its transpose

$P \in \mathbb{R}^{n\times n}$

Show $P^{-1} = P^T$ which is equivalent to showing $PP^T = I$

Using matrix multiplication definition,

$$(PP^T)_{ij} = \sum_{k=1}^{n} P_{ik} \cdot P^T_{kj} = \sum_{k=1}^{n} P_{ik} (P_{jk})$$

$(P^T)_{ij} = P_{ji}$ transpose definition

If $i=j$, it is a dot product on itself.

$$(PP^T)_{i=j} = \sum_{k=1}^{n} P_{ik} \cdot P_{ik} = (1\times 1) + (0 \times 0) + \cdots = 1$$

If $i \neq j$, we can verify that the dot product of those rows and columns is 0 because there is only one 1 per row and column, so the dot product "misses" the entry with 1.

Thus, the diagonal is filled with all ones and elsewhere is 0, which is the identity.

4. Show

$$PL_{ij}(\theta) = \begin{cases} L_{kj}(\theta)P & \text{if } i=\ell \\ L_{ij}(\theta)P & \text{otherwise} \end{cases} \qquad k > \ell \geq i$$

The key here is noticing that P is a symmetric matrix when only 2 rows are swapped in the identity and for a matrix $A \in \mathbb{R}^{n \times n}$, AP swaps rows k, j and PA swaps columns k, j since $P = P^T$.

case $i = \ell$:

Since $i = \ell$, the nonzero θ entry of $L_{\ell j}(\theta)$ is located in row ℓ. When applying $PL_{ij}(\theta)$, we swap rows k and ℓ such that θ gets moved from row ℓ to row k, denoted $L_{kj}(\theta)$ now. When applying $L_{kj}(\theta)P$, we swap columns k and ℓ in $L_{kj}(\theta)$ which has no effect on column j, so θ remains in that column, though the identity structure is still modified in the same manner as $PL_{ij}(\theta)$ since there is only one 1 in each row and column, so it is equivalent to swapping rows i and ℓ. Thus,

$$PL_{ij}(\theta) = L_{kj}(\theta)P \quad , \; i = \ell$$

case $i \neq \ell$:

Given our constraint $\ell > i$, the swaps occur below the row with θ, so swapping rows k and ℓ is equivalent to the original matrix $L_{ij}(\theta)$ with the columns k and ℓ swapped with θ still in its original position. Thus,

$$PL_{ij}(\theta) = L_{ij}(\theta)P \quad , \text{ otherwise}$$

In PLU factorization, the order of multiplication in $L^{-1}P^{-1}A = U$ is perform a swap, then elimination in this sequence:

$$L_{n-1}P_{n-1} \cdots L_1 P_1 A = U$$

Because we proved that $PL_k = L_k P$ where any nonzero, off diagonal entry is permuted with the swap, then we can substitute back in

$$L_{n-1}(L_{n-2}P_{n-1})P_{n-2} \cdots (L_3 P_2)(L_2 P_1) \Rightarrow L_{n-1}L_{n-2} \cdots L_2 L_1 P_{n-1}P_{n-2} \cdots P_2 P_1$$

Then, if we combine the final L and P matrices, we get

$$L_{total}P_{total}A = U \quad \Rightarrow \quad PA = LU \quad , \; L = L_{total}^{-1} \text{ and } P = P_{total}$$

5. Let $A \in \mathbb{R}^{n \times n}$

We can define a failure in partial pivoting if for the current pivot entry, every entry below it is exactly 0. To prove that A is singular, or not invertible, it is sufficient to show that the rows of A are linearly dependent or the determinant is 0.

If every entry at column k for $i \leq k$, which includes the diagonal, is zero, then the column span of $1 \leq j \leq k$ is linearly dependent since column k does not add any new dimension to the matrix since there are no nonzero entries in a new row position. This means $\text{Rank}(A) < n \Leftrightarrow$ linearly dependent $\Leftrightarrow$ singular

6. in code below

```python
def solve_ge(A_in, b_in, use_pivoting=True):
    A = A_in.copy().astype(float)  # non-destructive
    b = b_in.copy().astype(float)  # non-destructive
    n = len(b)

    # Gaussian Elimination
    for i in range(n - 1):
        if use_pivoting:
            # Partial Pivoting: Find the largest element in current column
            pivot_row = np.argmax(np.abs(A[i:, i])) + i
            A[[i, pivot_row]] = A[[pivot_row, i]]
            b[[i, pivot_row]] = b[[pivot_row, i]]

        for k in range(i + 1, n):
            l = A[k, i] / A[i, i]
            for m in range(i, n):
                A[k, m] -= l * A[i, m]
            b[k] -= l * b[i]

    # Back Substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        S = b[i]
        for j in range(i + 1, n):
            S -= A[i, j] * x[j]
        x[i] = S / A[i, i]

    return x


# Define input
eps = np.finfo(float).eps
A = np.array([[eps, -1, 0],
              [-1,  2, -1],
              [0,  -1,  2]])

b = np.array([eps - 1, 0, 1])

# Run both versions
x_no_pivot = solve_ge(A, b, use_pivoting=False)
x_with_pivot = solve_ge(A, b, use_pivoting=True)
```

```python
# Define input
eps = np.finfo(float).eps
A = np.array([[eps, -1, 0],
              [-1,  2, -1],
              [0,  -1,  2]])

b = np.array([eps - 1, 0, 1])

# Run both versions
x_no_pivot = solve_ge(A, b, use_pivoting=False)
x_with_pivot = solve_ge(A, b, use_pivoting=True)

print(f"True solution: [1, 1, 1]")
print(f"No pivoting:   {x_no_pivot}")
print(f"With pivoting: {x_with_pivot}")

''' TERMINAL OUTPUT
True solution: [1, 1, 1]
No pivoting:   [1. 1. 1.]
With pivoting: [1. 1. 1.]
'''

```